

PROYECTO FINAL • CS3014 • UTEC

Ray tracing acelerado con BVH

Estudio comparativo de heurísticas de construcción

SAH • Median • Morton • Intel Embree

Kalos Lazo • Marcelo Chinchá • Lenin Chávez

Universidad de Ingeniería y Tecnología • 2026-1

Agenda

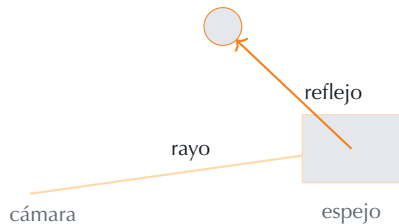
1. Introducción
2. Nuestro enfoque
3. El problema y la idea
4. Conceptos clave
5. Construcción del árbol
6. Recorrido y pipeline
7. Complejidades
8. Resultados
9. Conclusiones

SECCIÓN 1

Introducción

¿Qué es el ray tracing?

- ▶ Técnica de renderizado que **simula el camino de la luz**: por cada píxel se lanza un *rayo* desde la cámara.
- ▶ El rayo **rebota**, genera **sombras** y **reflejos** siguiendo leyes ópticas $\Rightarrow$ imagen fotorrealista.
- ▶ Es el estándar en cine de animación y, hoy, también en videojuegos (*NVIDIA RTX*, PS5).



¿Por qué importa?

Dónde se usa hoy el ray tracing

- ▶ **Cine / animación:** Pixar, Disney, ILM (toda película CG moderna).
- ▶ **Videojuegos:** *Cyberpunk 2077*, *Minecraft RTX* — iluminación global en tiempo real.
- ▶ **Diseño y arquitectura:** previsualización fotorrealista.
- ▶ **Ciencia e IA:** radar, simulación, *neural rendering*.

El reto

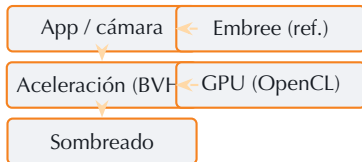
Es **computacionalmente muy caro**. Su viabilidad depende de *estructuras de datos* que lo aceleren → **el BVH**.

SECCIÓN 2

Nuestro enfoque

Nuestro enfoque

- ▶ Ray tracer **por software en CPU**, multihilo (4+ workers).
- ▶ Aceleración: **BVH propio** con SAH / Median / Morton (seleccionable en vivo).
- ▶ **Dos BVH** (estático + dinámico) — ver siguiente.
- ▶ Sombras, reflejos recursivos, iluminación indirecta (NEE).
- ▶ Backend **GPU OpenCL** opcional.
- ▶ Referencia industrial: **Intel Embree 4**.



Dos BVH: estático + dinámico

BVH estático — SAH

- ▶ Contiene la **escena fija** (edificios, calles).
- ▶ Se construye **una sola vez** con SAH (mejor calidad).
- ▶ Se reutiliza en todos los frames.

BVH dinámico —

- ▶ Objetos que **se mueven** (peatreros/peatones, meshes).
- ▶ Se **reconstruye cada frame** $\Rightarrow$ Morton (build rapidísimo).
- ▶ Pocos triángulos $\Rightarrow$ calidad suficiente.

Cada rayo consulta **ambos** árboles y se queda con el golpe más cercano.
Idea clave: la heurística debe casar con la frecuencia de cambio de la geometría.

SECCIÓN 3

El problema y la idea

El problema: el ray tracing ingenuo es intratable

El rayo se prueba contra **todos** los triángulos:

$$T_{\text{frame}} \approx \underbrace{P}_{\text{píxeles}} \cdot \underbrace{N}_{\text{triángulos}} \cdot (\text{rayos secundarios})$$

Un número que asusta

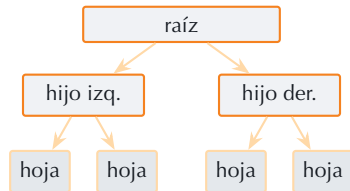
1280×720 con 50 000 triángulos $\approx 4,6 \times 10^{10}$ tests por frame $\Rightarrow \sim \mathbf{46 \text{ s por frame}}$.
Imposible en tiempo real.

- ▶ Necesitamos **descartar** grandes grupos de triángulos con pruebas baratas.
- ▶ $\Rightarrow$ una estructura de aceleración: el **BVH**.

BVH: la idea

- ▶ Árbol binario de **cajas alineadas a los ejes** (AABB).
- ▶ La raíz envuelve toda la escena; cada nodo **parte** sus triángulos en 2 hijos.
- ▶ Las **hojas** guardan pocos triángulos.
- ▶ El rayo prueba primero las cajas y solo desciende por las que golpea.

Costo por rayo: $O(N) \rightarrow \sim O(\log N)$.



SECCIÓN 4

Conceptos clave

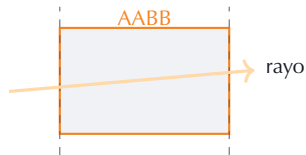
AABB y el test de los *slabs*

AABB: caja alineada a los ejes, con 2 esquinas (min, max).

Slab: banda entre 2 planos paralelos; una caja = 3 slabs (X, Y, Z).

El rayo entra en los 3 slabs a la vez sii:

$$t_{\min} = \max(\text{ent.}) \leq t_{\max} = \min(\text{sal.}) \Rightarrow \text{golpea}$$



Optimización clave

Se precalcula $\text{inv} = 1/\mathbf{D}$ por rayo $\Rightarrow$ luego solo se multiplica (división $\sim 20\times$ más lenta).

Möller–Trumbore: rayo – triángulo

Iguala el rayo $\mathbf{P} = \mathbf{O} + t\mathbf{D}$ con el triángulo $\mathbf{P} = \mathbf{v}_0 + u\mathbf{e}_1 + v\mathbf{e}_2$ y resuelve (t, u, v) :

```
vec3 e1 = v1 - v0, e2 = v2 - v0;
vec3 h = cross(d, e2);
float a = dot(e1, h);          if (fabs(a) < EPS) return false;
float f = 1.0f / a;
float u = f * dot(o - v0, h);   if (u < 0 || u > 1) return false;
float v = f * dot(d, cross(o-v0, e1)); if (v < 0 || u+v > 1) return false;
t = f * dot(e2, cross(o-v0, e1)); return t > EPS;
```

$u, v \geq 0$ y $u+v \leq 1$ son la prueba de *dentro del triángulo*; además (u, v) son los **pesos baricéntricos** para interpolar color, normal y textura.

SECCIÓN 5

Construcción del árbol

Construcción *top-down*

1. Se parte con **todos** los triángulos en la raíz.
2. Centroide de cada uno: $(v_0 + v_1 + v_2)/3$.
3. Mientras un nodo tenga > 2 triángulos:
 - eje de mayor extensión de centroides;
 - plano de corte $\Rightarrow$ *depende de la heurística*;
 - reparto por centroide en 2 hijos.
4. Recursión hasta hojas de ≤ 2 triángulos.

La pregunta del proyecto

¿**Dónde** cortar? Comparamos 3 heurísticas — y de eso vive el *benchmark*.

Tres heurísticas de construcción

SAH



Minimiza el **costo** = $\text{área} \times \text{n.º de triángulos}$. 12 cubos (*binning*).

Mejor árbol; build más lento.

Median

Corta por la **mediana** de centroides (mitad/mitad).

Simple; ignora el tamaño.

Morton

Ordena por **código Z** (bits intercalados) y corta por la mitad del índice.

Build rapidísimo; árbol mediocre.

Comparten centroides, eje y recursión; solo cambia **dónde cortar**.

SAH: heurística de área de superficie

Intuición: un rayo es más probable de entrar en una caja más grande.
 $P(\text{entrar en hijo}) \propto \text{área}(\text{hijo})/\text{área}(\text{padre})$. Costo esperado:

$$C_{\text{split}} = 1 + \frac{A_L n_L + A_R n_R}{A_{\text{padre}}}$$

Binning: en vez de probar todos los cortes $O(N^2)$, se dividen los centroides en 12 cubos y se prueban solo 11 cortes $\Rightarrow$ build $O(N \log N)$, casi óptimo.



SECCIÓN 6

Recorrido y pipeline

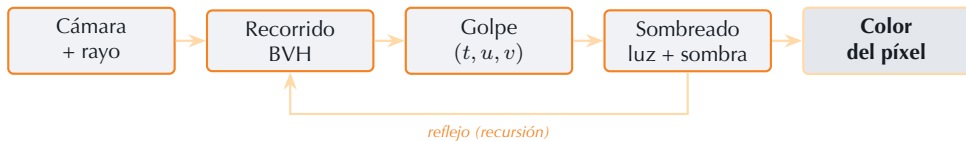
Recorrido (*traversal*) con pila y poda

- ▶ Recorrido **iterativo** con pila fija (no recursión).
- ▶ **Poda 1:** caja no golpeada $\Rightarrow$ se salta el subárbol.
- ▶ **Poda 2:** caja más lejos que el mejor $t \Rightarrow$ se salta.
- ▶ En la hoja: Möller–Trumbore a sus pocos triángulos.

Un rayo que tocaría miles de triángulos acaba probando **solo unos pocos**.

```
stack.push(raiz);
while (pila no vacia) {
    nodo = pop();
    if (!aabb_hit(nodo.box, inv, best)) continue;
    // poda
    if (es_hoja) test_tris(nodo);
    else        push(hijos);
}
```

Pipeline completo



- ▶ El sombreado puede **generar nuevos rayos** (sombra, reflejo) $\Rightarrow$ recursión.
- ▶ La imagen se reparte en **franjas** entre los núcleos (multihilo, semáforos SDL).

SECCIÓN 7

Complejidades

Complejidades del BVH

Operación	Complejidad	Nota
Build SAH (binning)	$O(N \log N)$	12 cubos por nodo, casi óptimo
Build Morton	$O(N \log N)$	domina el <i>sort</i>
Build Median	$O(N \log N)$	quickselect $O(N)$ por nivel
Recorrido (promedio)	$O(\log N)$	descarta subárboles
Recorrido (peor caso)	$O(N)$	geometría degenerada
Memoria	$O(N)$	nodos + triángulos
Rayo – AABB (slab)	$O(1)$	3 ejes
Rayo – triángulo	$O(1)$	Möller–Trumbore

$N = \text{n.º de triángulos}$. La calidad del árbol cambia la *constante* del recorrido, no la asintótica.

SECCIÓN 8

Resultados

Setup experimental

Escena: ciudad procedural (fachadas con ventanas):

- ▶ Small: **14 896** triángulos
- ▶ Medium: **18 468**
- ▶ Large: **31 644**

Plataforma: Apple M5, 10 núcleos (4P+6E), -O3 -march=native.

Referencia: Intel Embree 4 sobre la misma geometría.

Metodología (reproducible: --bench):

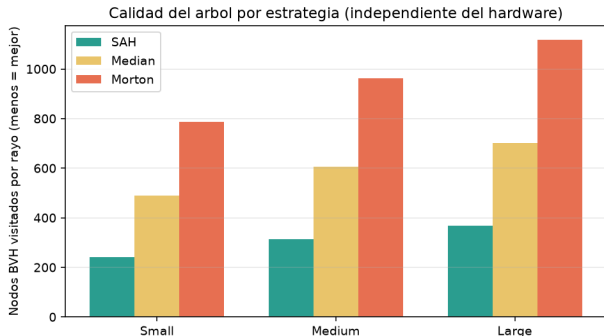
- ▶ 5 frames de *warmup* + promedio de 20.
- ▶ Traversal puro (1 hilo, sin sombreado).
- ▶ **Nodos/rayo:** visitas por rayo — métrica hardware-independiente.

Resultado principal — escena Large (31 644 triángulos)

Estrategia	Build (ms)	Render (ms)	Trav. (ms)	Nodos/rayo
SAH (nuestra)	6,4	112	79	367
Median	5,0	199	151	701
Morton	2,6	288	242	1 119
Embree (referencia)	—	—	33	—

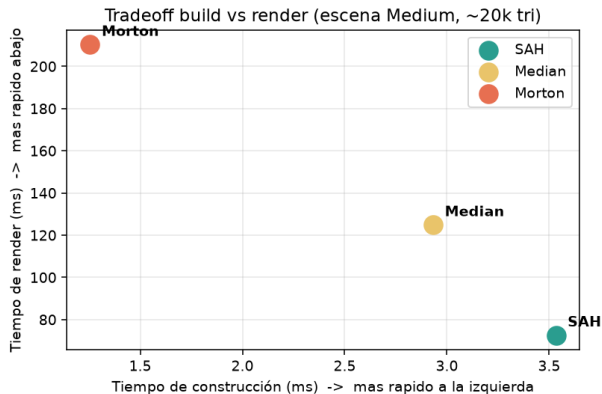
- ▶ SAH visita $\sim 3\times$ **menos nodos** que Morton $\Rightarrow$ render $\sim 2,6\times$ **más rápido**.
- ▶ Embree recorre $\sim 2,4\times$ más rápido que nuestra SAH (kernels SIMD).

Calidad del árbol: SAH domina



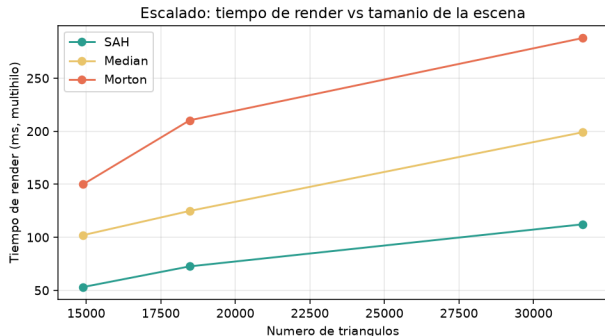
- ▶ Métrica **nodos/rayo**: independiente del hardware.
- ▶ SAH es la más baja *en todas las densidades*.
- ▶ En Large: **SAH 367 vs Morton 1 119**.

Compromiso *build* vs calidad



- ▶ Esquina abajo-izquierda = ideal.
- ▶ Morton: build rápido, render lento.
- ▶ SAH: build más lento, render mucho más rápido.
- ▶ Se construye *1* vez y se recorre *millones* $\Rightarrow$ **compensa SAH.**

Escalado con el tamaño de escena



- ▶ Todas son $O(\log N)$, pero la *constante* difiere.
- ▶ SAH degrada mucho más lento al crecer la escena.
- ▶ Morton se va $\sim 3\times$ por encima en Large.

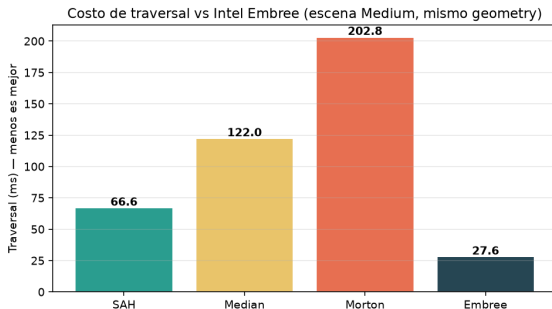
Cómo funciona el BVH de Intel Embree

- ▶ **MBVH**: nodos de **4 u 8 vías** (no binario). Un paso ramifica más.
- ▶ **SIMD**: con una sola instrucción AVX/NEON prueba **4–8 cajas a la vez**.
- ▶ **Layout** depth-first con hijos implícitos ⇒ mejor caché.
- ▶ Nodos **cuantizados**: menos memoria, menos fallos de caché.
- ▶ Opcional: *ray stream* (varios rayos a la vez).

Por qué es más rápido

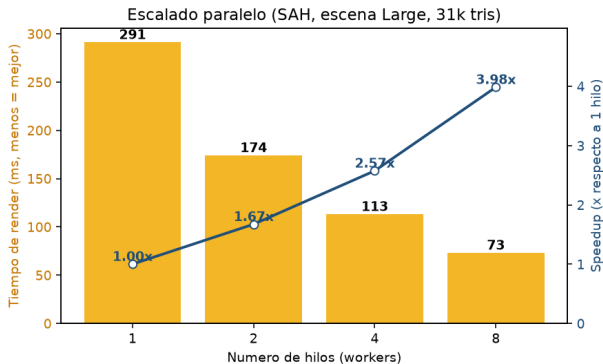
- ▶ **No** por mejor árbol (nuestra SAH es comparable).
- ▶ Sí por la **ingeniería del kernel** (SIMD + MBVH + caché).
- ▶ Lección: cerrar la brecha requiere ingeniería de kernels, *no* un mejor árbol.

Comparación con Intel Embree



- ▶ Misma geometría: Embree ~ 33 ms vs nuestra SAH ~ 79 ms.
- ▶ $\sim 2,4\times$ más rápido.
- ▶ Embree = *término de comparación*, no competidor.
- ▶ Valida que estamos en el orden de magnitud correcto.

Escalado paralelo (más benchmarks)



- ▶ SAH/Large, de 1 a 8 *workers*.
- ▶ Render: **291 ms** → **73 ms**.
- ▶ **Speedup** $\sim 4\times$ al usar los 10 núcleos (4P+6E).
- ▶ El paralelismo por franjas escala muy bien.

SECCIÓN 9

Conclusiones

Conclusiones

- ▶ Ray tracer CPU completo con BVH y **benchmark reproducible** (`--bench`, `--scaling`).
- ▶ Comparamos **3 heurísticas** + referencia **Embree** sobre la misma geometría.
- ▶ **Hallazgo:** la **SAH** da el mejor árbol ($\sim 3\times$ menos nodos/rayo que Morton) $\Rightarrow$ render $\sim 3\times$ más rápido.
- ▶ La brecha con Embree está en el *recorrido SIMD* (MBVH), no en la estructura.
- ▶ El paralelismo por franjas escala $\sim 4\times$ con los núcleos disponibles.

Demo en vivo

Demo en tiempo real

```
./bvh_raytracer --width 1280 --height 720
```

TAB raytrace • B fuerza bruta • V cajas • M menú

- ▶ Alternar SAH / Median / Morton en vivo y ver el cambio de FPS.
- ▶ Comparar BVH vs fuerza bruta $O(N)$ (tecla B).
- ▶ Visualizar las cajas del árbol (tecla V).

Trabajo futuro

- ▶ Recorrido **SIMD** (NEON): varios rayos/hijos por instrucción.
- ▶ **MBVH**: nodos de 4 vías.
- ▶ Layout **SoA** y nodos cuantizados.
- ▶ Construcción **paralela** del árbol.
- ▶ Comparar con **SBVH** (split) para geometría que se intersecta.
- ▶ Más escenas y resoluciones.

¡Gracias!

Preguntas y respuestas

Ray tracing acelerado con BVH • UTEC • CS3014